

Write-Up on Bayesian Optimization

Advanced Methods in Applied Statistics 2026

Jeppe Peder Grejs Petersen (rpx928), Asger Thormann (kfw373),
Niels August Davidsen (phx657), Jeppe Brøndum Bach (sdq405)

Introduction

In the modern world of scientific computing, optimizing model parameters becomes an increasingly difficult task as models grow larger and become more costly to run. Both climate models and machine learning models are excellent examples of this problem; running the model with a single set of parameters can be costly in both time and resources (see for example Mrozowska et al., 2025), and the wrong parameters can have large consequences for the model performance. Therefore, it is desirable to have an optimization algorithm that evaluates the model as few times as possible using informed parameter estimates.

Bayesian Optimization (BO) is a method presented by Mockus et al., 1978 to find a global extremum of an unknown function $f(\mathbf{x})$, in the following this will be a minimum. In the context of optimization of model parameters, $\mathbf{x}$ is a point in parameter space. Finding the minimum of $f(\mathbf{x})$ is done through an iterative process where one uses a prior and evaluates this using an acquisition function. Maximizing the acquisition function, one obtains a new $\mathbf{x}$ which can be evaluated in f to give a data point and constrain the problem. The prior is then updated based on the data point and the loop continues. Unlike gradient-based optimization, BO uses all previously evaluated point, to suggest a new point. It is up to the user to decide on a meaningful prior and acquisition function fit for the problem at hand. A small visual example of how a Bayesian Optimization algorithm works is shown in Figure 1.

In the paper *Practical Bayesian Optimization of Machine Learning Algorithms* by Snoek et al., 2012 BO is used in the scope of machine learning hyper parameter optimization. Using the Gaussian Process prior they explore and benchmark different choices of acquisition function on three recently published machine learning problems. Additionally they implement parallelization of BO by using Markov Chain Monte Carlo (MCMC) sampling of pending function evaluations.

In our short write up we will go over the basic concepts of Bayesian Optimization as presented by Snoek et al. (2012). This includes an explanation of the Gaussian Process prior and the Expected Improvement (EI) acquisition function.

Gaussian Process prior

Snoek et al., 2012 use a Gaussian Process (GP) prior in their Bayesian Optimization. A Gaussian Process is a distribution of functions determined by a mean, $\mu(\mathbf{x})$, and a kernel

$k(\mathbf{x}, \mathbf{x}')$, where again $\mathbf{x}$ is a point in parameter space and $\mathbf{x}'$ are the other points that might have influence on the value of $f(\mathbf{x})$ (Rasmussen & Williams, 2006). The functions drawn from the Gaussian Process, $g(\mathbf{x}) \sim \mathcal{GP}(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$, are multivariate gaussian distributions of points with mean $\mu(\mathbf{x})$ and with covariance and variance determined by the kernel $k(\mathbf{x}, \mathbf{x}')$. The role of the kernel is to smooth the signal through introducing covariance between points. When using GP as a prior the function to be minimized, $f(\mathbf{x})$, is assumed to be contained in the Gaussian Process distribution.

The kernel variance and the mean $\mu(\mathbf{x})$ also enter as hyperparameters in the optimization process. However, the kernel type i.e. the smoothing degree is chosen (Snoek et al., 2012). Snoek et al., 2012 advocates for the ARD Matérn 5/2 kernel (see Appendix 1). The choice of kernel determines what types of functions will be included in the posterior by adjusting the correlation between points that are close in parameter space. Little smoothing and the posterior will include possible minima close to already evaluated points. More smoothing and the posterior will not vary close to the evaluated points. This is a trade off between the Bayesian Optimization finding narrow minima but potentially using more function calls to search small areas in hyperparameter space and the Bayesian Optimization exploring more of the hyperparameter space faster but potentially missing small local minima.

The choice by Snoek et al., 2012 to use the Matérn 5/2 kernel is presented as a choice to favor more ragged functions than what is usually done in Bayesian Optimization.

Figure 1 shows an example of Bayesian Optimization with a Matérn 5/2 kernel. The gray functions are sampled randomly from the Gaussian Process distribution, and the orange line is $m(\mathbf{x})$. The figure shows the iterative forming of a posterior distribution that becomes the prior in the next iteration.

Acquisition Function

The initial idea of the Bayesian Optimization is to propose new points for evaluations that are likely to improve. Under the assumption that new points are very expensive to calculate, more computational power can be spend in proposing expected valuable points. In this process the acquisition function is the key component. In the optimization loop a new point is proposed as $x_{next} = \operatorname{argmax}(a(\mathbf{x}))$, where $a(\mathbf{x})$ is the acquisition function. There are multiple options for the choice of $a(\mathbf{x})$ depending on optimization strategy. The acquisition function is written in a form to maximize expected

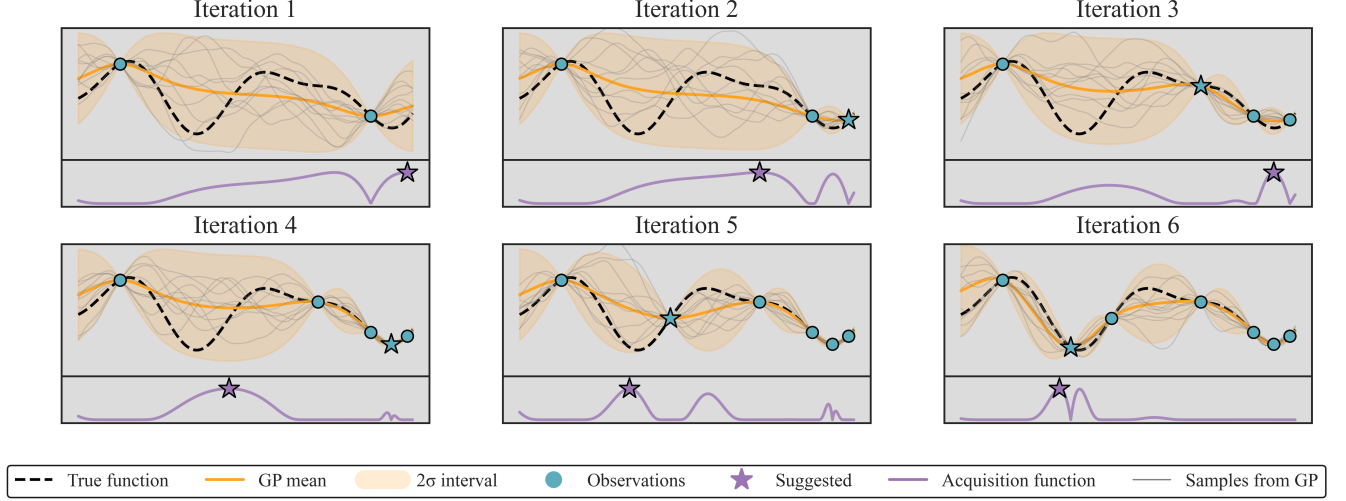


Figure 1: A visualization of the Bayesian Optimization loop. In iteration 1, two points have already been evaluated, and the Gaussian process (GP) has been updated accordingly. To see how the covariance matrix is updated, see Appendix . Note how the algorithm suggests points away from the local minima, once it has explored this

improvement:

$$a_{EI}(\mathbf{x}; \{\mathbf{x}_n, y_n\}, \theta) = \sigma(\mathbf{x}; \{\mathbf{x}_n, y_n\}, \theta) \cdot \left(\gamma(\mathbf{x}) \Phi(\gamma(\mathbf{x})), \right. \\ \left. + \mathcal{N}(\gamma(\mathbf{x}); 0, 1) \right) \quad (1)$$

In Eq. 1 Φ denotes the CDF of the standard normal distribution, $\mathcal{N}$ the normal PDF, $\{\mathbf{x}_n, y_n\}$ the previous observations, and θ the GP hyper-parameters. Here, Φ represents the probability of improvement over the current best value, while $\mathcal{N}$ accounts for the magnitude of potential improvement due to predictive uncertainty. The function γ is defined as:

$$\gamma(\mathbf{x}) = \frac{f(\mathbf{x}_{best}) - \mu(\mathbf{x}; (\mathbf{x}_n, y_n), \theta)}{\sigma(\mathbf{x}; (\mathbf{x}_n, y_n), \theta)} \quad (2)$$

Where the function value of the best point so far is compared to the predictive mean, $\mu(\mathbf{x}; (\mathbf{x}_n, y_n), \theta)$ and $\sigma(\mathbf{x}; (\mathbf{x}_n, y_n), \theta)$ is the variance, both from the Bayesian posterior found by constraining the Gaussian Process prior with the points in which the function is evaluated. Using Eq. 2 the acquisition function is written in a form to maximize expected improvement:

This equation balances exploration and exploitation. If the variance is small and the predictive mean is larger than the best functional value evaluated so far the expected improvement is low and the algorithm will not choose points in this region. In this way exploration and exploitation is balanced in Eq. 1.

If γ is large as of Eq. 2, a_{EI} is also large and thus if the predictive mean is better than our current best evaluated point there is an expected improvement.

Thus this method favors regions with high variance and regions with low predictive mean. In Figure 1 the purple line displays the acquisition function, which changes for each iteration. It is seen that regions of high variance yields a large

acquisition function value and similarly that regions with low predictive mean are favored too.

There are several ways to implement the acquisition function and the paper (Snoek et al., 2012) introduces both probability of improvement and upper confidence bound as methods. But as Snoek et al., 2012 focuses on expected improvement, so does this write up.

Furthermore Snoek et al., 2012 discusses that evaluation of points varies in time depending on the function to evaluate, e.g. numbers of layers changes the time it takes to train a neural network and therefore the time to evaluate the function. Thus a metric to account for in the acquisition function could also be the expected improvement per second. As the goal of Bayesian optimization is to limit computational time, minimizing wall-clock time naturally follows and the implications are explored by the authors.

In modern day training and simulation can be parallelized on High Performance Parallel Computers (HPPC's) and thus new points need to be suggested while other calculations are running but without repeating the same calculations. The authors solve this by sampling possible outcomes of pending evaluations and averaging these to use as if they were observations.

Conclusion

Bayesian Optimization is a way to optimize model parameters especially useful in cases where the model is computationally heavy to evaluate. The algorithm takes a Bayesian approach by assuming a prior and then updating it using data that is selected with an acquisition function. Snoek et al., 2012 uses a Gaussian Process prior with a Matérn 5/2 kernel and their acquisition function maximizes the expected improvement. In doing this, they produce better results than by using the current state of the art optimization methods (Snoek et al., 2012).

References

- Mrozowska, M., Avery, J., Stoustrup, A., Nuterman, R., Johnsen, C., Thormann, A., & Jochum, M. (2025). Bayesian optimization with gpu acceleration for ocean models. *Journal of Geophysical Research: Machine Learning and Computation*, 2. <https://doi.org/10.1029/2024JH000517>
- Mockus, J., Tiesis, V., & Zilinskas, A. The application of bayesian methods for seeking the extremum. In: *Towards global optimization*. Elsevier, 1978, pp. 117–129.
- Snoek, J., Larochelle, H., & Adams, R. P. Practical bayesian optimization of machine learning algorithms (F. Pereira, C. Burges, L. Bottou, & K. Weinberger, Eds.). In: *Advances in neural information processing systems* (F. Pereira, C. Burges, L. Bottou, & K. Weinberger, Eds.). Ed. by Pereira, F., Burges, C., Bottou, L., & Weinberger, K. 25. Curran Associates, Inc., 2012. https://proceedings.neurips.cc/paper_files/paper/2012/file/05311655a15b75fab86956663e1819cd-Paper.pdf
- Rasmussen, C. E., & Williams, C. K. I. (2006). *Gaussian processes for machine learning*. MIT Press.

Appendix 1: Kernels

Snoek et al. uses the ARD Matérn 5/2 kernel given by

$$K_{M52}(\mathbf{x}, \mathbf{x}') = \sigma^2 \left[1 + \sqrt{5r^2(\mathbf{x}, \mathbf{x}') + \frac{5}{3}} r^2(\mathbf{x}, \mathbf{x}') \right] \exp(-\sqrt{5r^2(\mathbf{x}, \mathbf{x}') + \frac{5}{3}} r^2(\mathbf{x}, \mathbf{x}')). \quad (3)$$

Where $r^2(\mathbf{x}, \mathbf{x}') = \sum_{n=1}^D \frac{(x_d - x'_d)^2}{\theta_d^2}$, D is the number of parameters, x_d are the parameters, and θ_D is a length scale parameter (Snoek et al., 2012).

Appendix 2: Additional Figures

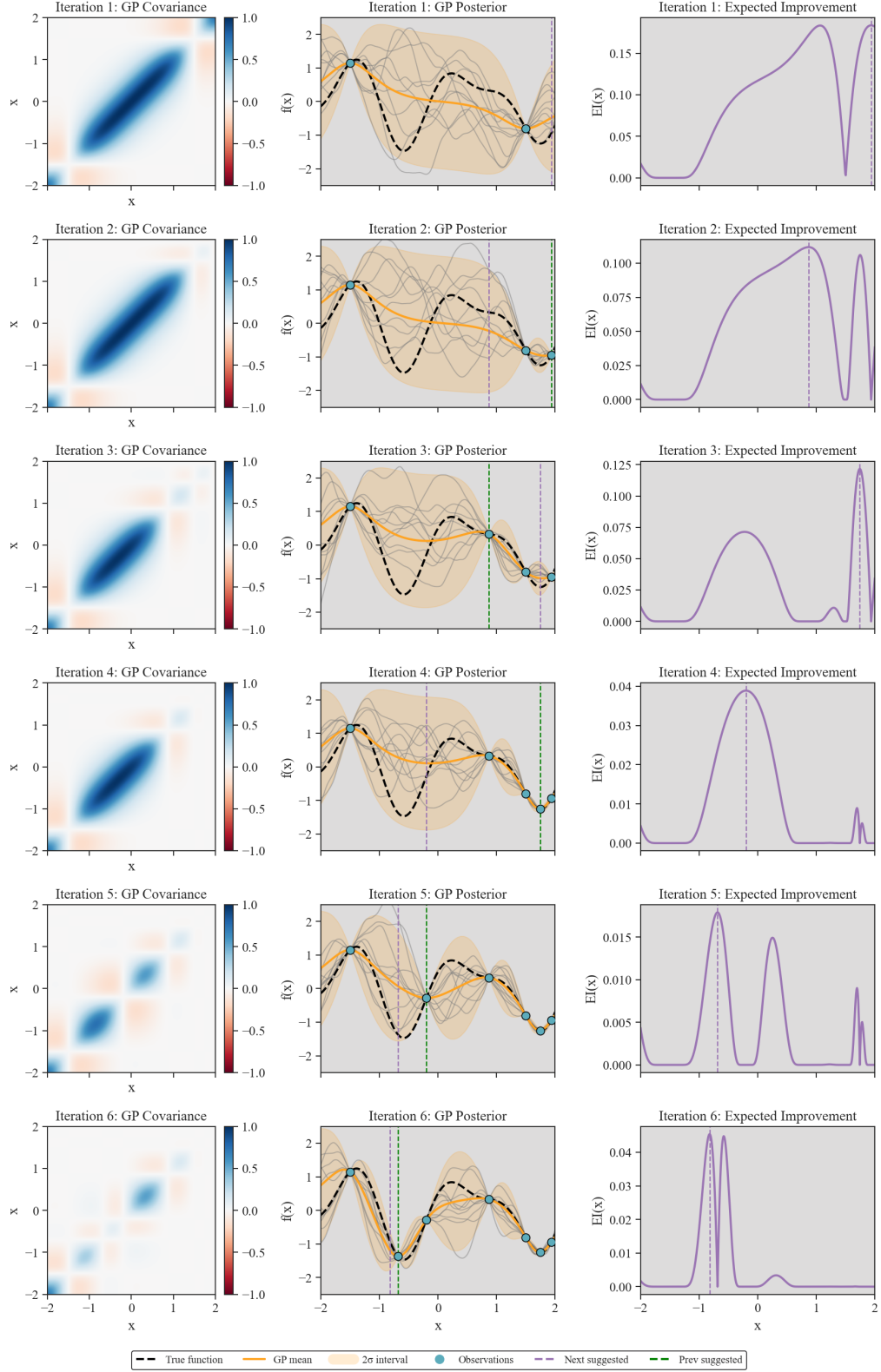


Figure 2: Same evaluated points as Figure 1. Here the covariance matrix used to draw points from the multivariate normal distribution is shown, and how including more points binds the solution.